Duke UNIVERSITY

Implementation and ramifications of TAMER-OP: Toolkit for Algebraic Module Encodings over $\mathbb{R}^n$ and Other Posets

Scope, flexibility, and efficiency in one system

Erik Mendes Novak
Mentor: Ezra Miller

April 9 2026

Thesis Defense

What is a persistence module?

A persistence module is a functor

$$M : Q \to \mathbf{Vect}_{\mathbb{k}},$$

where Q is a poset and $\mathbf{Vect}_{\mathbb{k}}$ is the category of vector spaces.

- Each parameter value $q \in Q$ carries a vector space $M(q)$.
- Each order relation $q \leq q'$ carries a linear map

$$M(q \leq q') : M(q) \to M(q').$$

- Functoriality $\longleftrightarrow$ commutativity

In ordinary persistence, Q is a chain. In multiparameter persistence, Q is often

$$\mathbb{Z}^n \quad \text{or} \quad \mathbb{R}^n$$

with the coordinatewise order.

That is the source of both the expressive power and the computational difficulty.

Why multiparameter persistence is hard

In one parameter, strong structure theorems lead to interval decompositions and barcodes.

In several parameters, three problems appear immediately:

1. the indexing poset is only partially ordered, not totally ordered;
2. there is no barcode-style complete classification in general;
3. analogs of intervals are infinitely or uncountably generated

So the practical question becomes:

> *How can we retain enough algebra to do meaningful computation, while still reducing the problem to finite data?*

The answer in TAMER-OP is *tameness* and *finite encoding*.

Tameness and finite encoding

A module is tame when it is constant on each of finitely many regions.

Finite encoding [Miller25]

A module M on a poset Q is finitely encoded by

- a finite poset P,
- a monotone classifier map $\pi : Q \to P$,
- and a P-module H,

such that the pullback satisfies

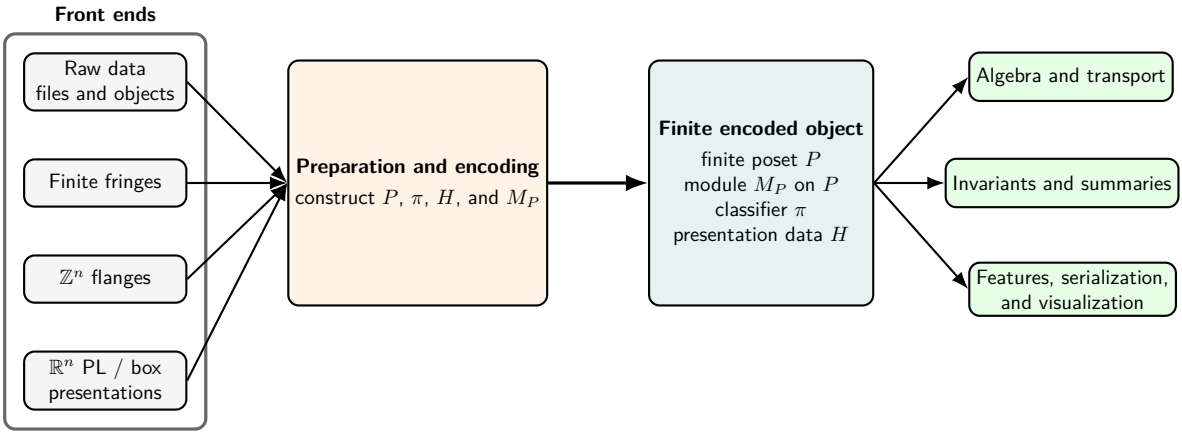
$$M \cong \pi^* H.$$

A fringe presentation expresses a module in terms of indicator modules for upsets and downsets, via a finite matrix.

This is the key computational move:

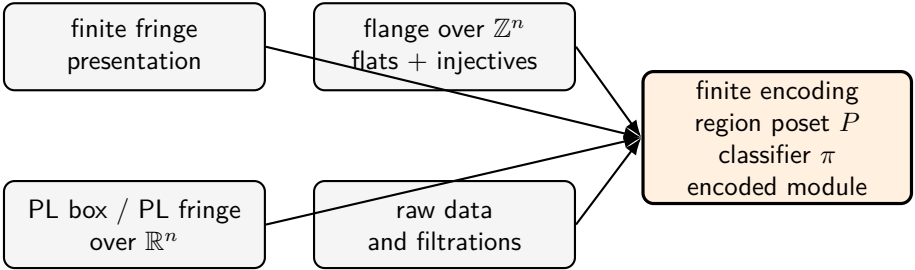
replace a large or infinite ambient parameter space by a finitely encoded module losslessly

The architecture: wide front end, thin center, wide output



This is the central claim of the software: heterogeneous inputs, one crucial encoding move, then many downstream options.

Four input modes, one internal language



This is why TAMER-OP is flexible: the user can begin from the representation that is natural for the problem, but all roads lead to the same finite encoded object.

Why encoding-first matters

Scope

One system for:

- raw data ingestion,
- $\mathbb{Z}^n$ and $\mathbb{R}^n$ front ends,
- finite encodings,
- exact algebra,
- invariants, geometry, and export.

Flexibility

Users do not have to commit to a single invariant too early.

The encoded module can be inspected algebraically before choosing rank, slice, image, or geometric summaries.

Efficiency

Encoding removes redundant ambient information while keeping what later computation needs.

That gives both tractability and speed.

Project scope at a glance

- **65000 lines of code** in just the source files
- **37 source files** in the present codebase snapshot.
- **4 front end input modes**: finite fringe, flange, PL box / PL fringe, and raw data.
- **130+ named invariant, distance, geometry, and support routines** in the current invariant layer.
- **100+ named algebraic and homological constructions** across exact-category, resolution, complex, and derived-functor layers.
- Support for $\mathbb{Z}^n$, $\mathbb{R}^n$, **and finite posets** inside the same workflow.

Why this matters

This is not a narrow experimental codebase built around one descriptor.

It is a broad computational environment for encoded multiparameter persistence.

Counts are taken from the project snapshot on 9 April 2026 and the exported invariant/algebra layers used in the thesis and companion decks.

What users can do after encoding: invariants

Family	Examples	Output object
Rank and dimension	restricted Hilbert, rank invariant, rectangle signed measures	vectors, rank tables, sparse signed data
Slice-based	slice barcodes, matching distance, sliced bottleneck / Wasserstein	barcodes, scalar distances
Function/image summaries	multiparameter landscape, MPPI	tensors, raster images
Euler and signed measures	Euler surface, Euler signed measure, point signed measure	grids, sparse signed point measures
Geometry-aware summaries	support measure, aspect ratio, interface measure, module geometry summary	scalars, histograms, weighted summaries
Support-theoretic	Betti/Bass support, socle, associated primes	finite algebraic descriptors

The key message: no single invariant is enough, so the software implements a family of invariant types specialized to different needs.

What users can do after encoding: algebra

Exact-category layer

- kernels, images, cokernels, quotients
- submodules and inclusions
- pullbacks, pushouts, equalizers, coequalizers
- short exact sequences and snake lemma

Resolution and derived layer

- projective covers and injective hulls
- indicator resolutions
- Hom, Ext, Tor
- long exact sequences

Complex-level layer over encoding poset P

- mapping cones and distinguished triangles
- RHom and derived tensor
- hyperExt and hyperTor
- spectral-sequence infrastructure

Why this is unusual

Most multiparameter workflows stop after constructing an invariant.

TAMER-OP keeps the module itself, so users can do algebra *before* they summarize.

Representative benchmark: full pipeline speedups over MULTIPERS

Invariant	Rips	Codensity rips	Rips lowerstar	Landmark	Alpha	Cubical
Euler	5.4	7.5	13.7	5.6	26.4	26.8
rank signed measure	∞	∞	∞	∞	∞	∞
slice barcodes	3.7	3.2	3.5	5.1	–	9.5
mp_landscape	4.8	2.7	5.5	6.0	–	11.2
restricted_hilbert	–	1.33	1.24	–	25.2	–
rank_invariant	∞	∞	∞	∞	∞	∞

Numbers are speedup factors for TAMER-OP over MULTIPERS on the supplied end-to-end ingestion-plus-invariant benchmarks.

- "∞" means no tractable MULTIPERS result on that benchmark.
- The important point is not just that TAMER-OP is faster on overlapping tasks, but that it also supports broader encoded workflows.

Why these benchmarks matter

Near-term message

Even on the tasks that other tools already attempt, the encoding-first approach is highly competitive and often much faster.

Long-term message

The point is not merely speed. Encoding creates a finite middle ground:

- enough compression to be tractable,
- enough structure to keep exact algebra alive.

raw ambient module
too large to compute on directly

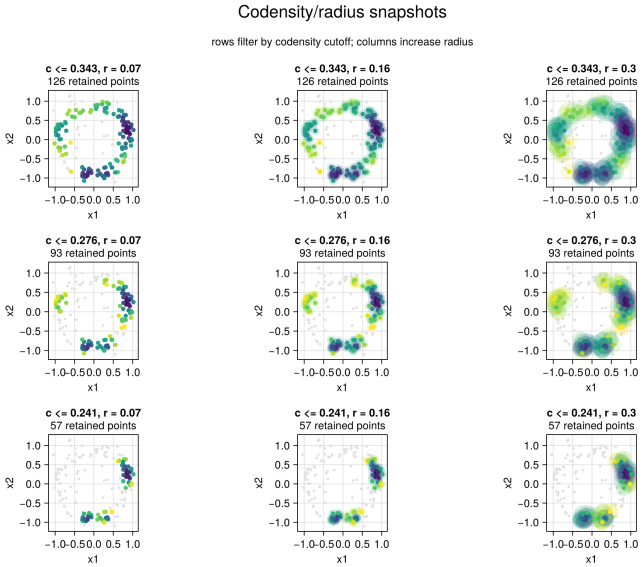
→

finite encoding
small enough to compute on

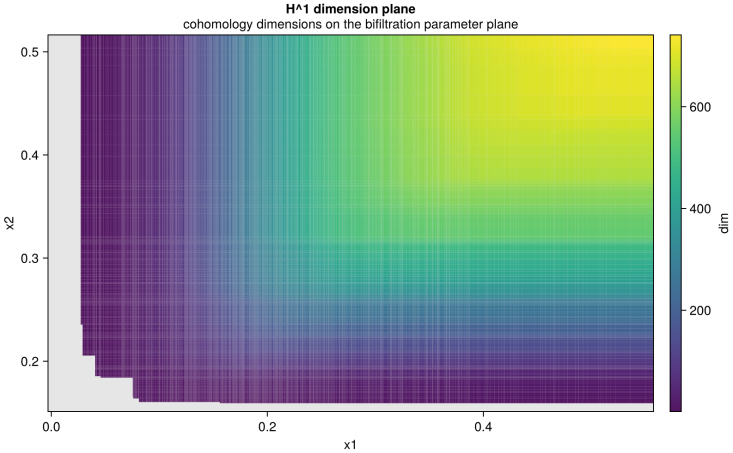
→

invariants, features,
and exact algebra

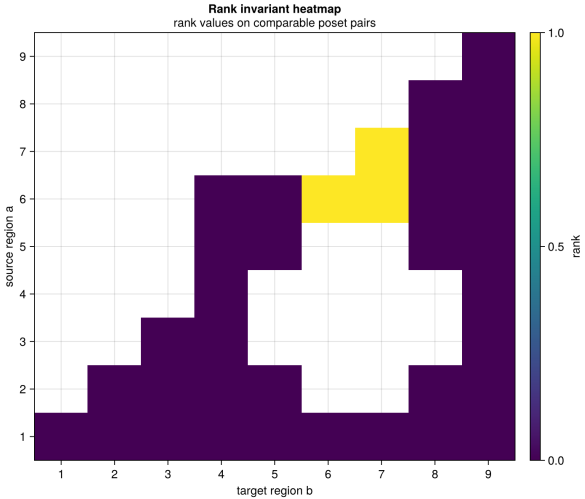
Visualization examples: Rips codensity bifiltration snapshot



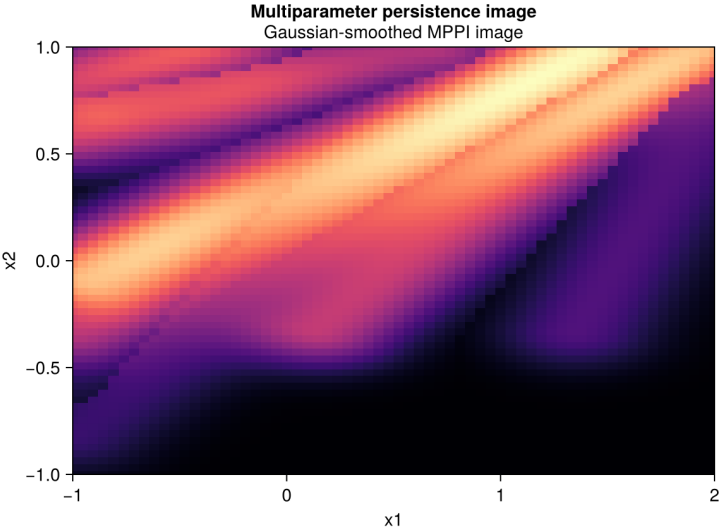
Visualization examples: Annulus H_1 support plane for Rips codensity



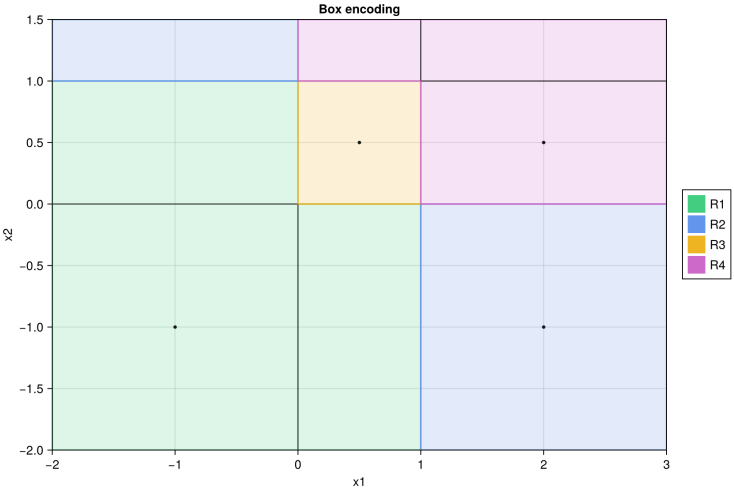
Visualization examples: Rank heatmap



Visualization examples: Visual of an MPP image



Visualization examples: Encoding of a PL box



What need does TAMER-OP fill?

- It treats finite encoding as the central computational move, not as an afterthought.
- It supports raw data, $\mathbb{Z}^n$ presentations, $\mathbb{R}^n$ presentations, and finite-poset modules in one framework.
- It keeps the module available for exact algebra, rather than forcing an invariant too early.
- It supports broad invariant families and geometry-aware summaries after encoding.
- On overlapping pipeline tasks, it is already highly competitive and often substantially faster than current alternatives.

Takeaway

If multiparameter persistence is going to become a practical algebraic technology rather than a collection of isolated summaries, then we need an encoding-first system. That is the niche TAMER-OP is built to fill.

Deployment and use

The software is already finding use in projects and natural interest in the community:

- Ramachandran plots and protein folding work with Dr. Michael Prisant and Dr. Jane Richardson at Duke
- Implementation discussions with Jan Jendrysiak of TU Munich, soon in Oxford
- I will road show it at SoCG 2026
- The repo already has 14 stars on Github
- 1-hour tutorial at Combinatorial Coworkspace in Kleinwalsertal, Austria, in March 2026
- Soon: interaction with David Loiseaux, creator of MULTIPERS

AI disclaimer

AI was used strictly for:

- Writing docstring for the code
- Guiding research direction, especially in invariants
- Creating TikZ diagrams
- Helping generate tests
- Writing boilerplate for benchmark harnesses
- Diagnosing Julia inefficiencies
- Writing the linear algebra backend
- Making the API user-friendly

All use of AI was strictly monitored and cross-checked.

Github

www.github.com/erikmnovak/TamerOp.jl

References

E. Miller.
Homological algebra of modules over posets.
arXiv:2008.00063, 2020.

L. Waas.
Notes on abelianity of categories of finitely encodable persistence modules.
arXiv:2407.08666, 2024.

E. Miller.
Stratifications of real vector spaces from constructible sheaves.
J. Appl. Comput. Topol., 2023.

M. Kashiwara and P. Schapira.
Persistent homology and microlocal sheaf theory.
JACT, 2018.

W. Crawley-Boevey.
Decomposition of pointwise finite-dimensional persistence modules.
J. Algebra Appl., 2015.

A. Zomorodian and G. Carlsson.
Computing persistent homology.
DCG, 2005.

References (continued)

G. Carlsson and A. Zomorodian.
The theory of multidimensional persistence.
DCG, 2009.

D. Cohen-Steiner, H. Edelsbrunner, and J. Harer.
Stability of persistence diagrams.
DCG, 2007.

H. Edelsbrunner, D. Letscher, and A. Zomorodian.
Topological persistence and simplification.
DCG, 2002.

M. Lesnick.
The interleaving distance on multidimensional persistence modules.
FoCM, 2015.

M. Lesnick and M. Wright.
Interactive visualization of 2-D persistence modules.
arXiv:1512.00180.

F. Chazal et al.
The Structure and Stability of Persistence Modules.
Springer, 2016.

References (continued)

S. Oudot.
Persistence Theory.
AMS, 2015.

H. Edelsbrunner and J. Harer.
Computational Topology.
AMS, 2010.

S. Mac Lane.
Categories for the Working Mathematician.
Springer, 1998.

C. Weibel.
An Introduction to Homological Algebra.
Cambridge, 1994.

D. Eisenbud.
Commutative Algebra.
Springer, 1995.

E. Miller and B. Sturmfels.
Combinatorial Commutative Algebra.
Springer, 2005.